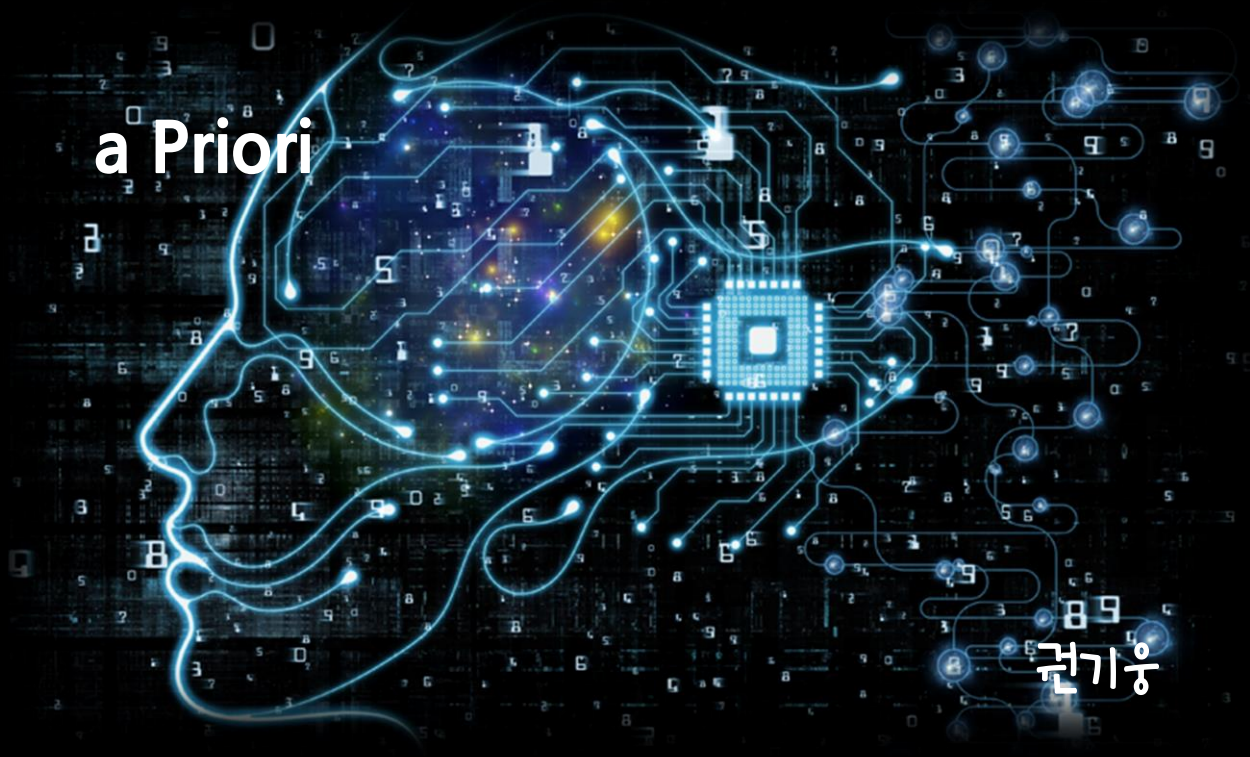
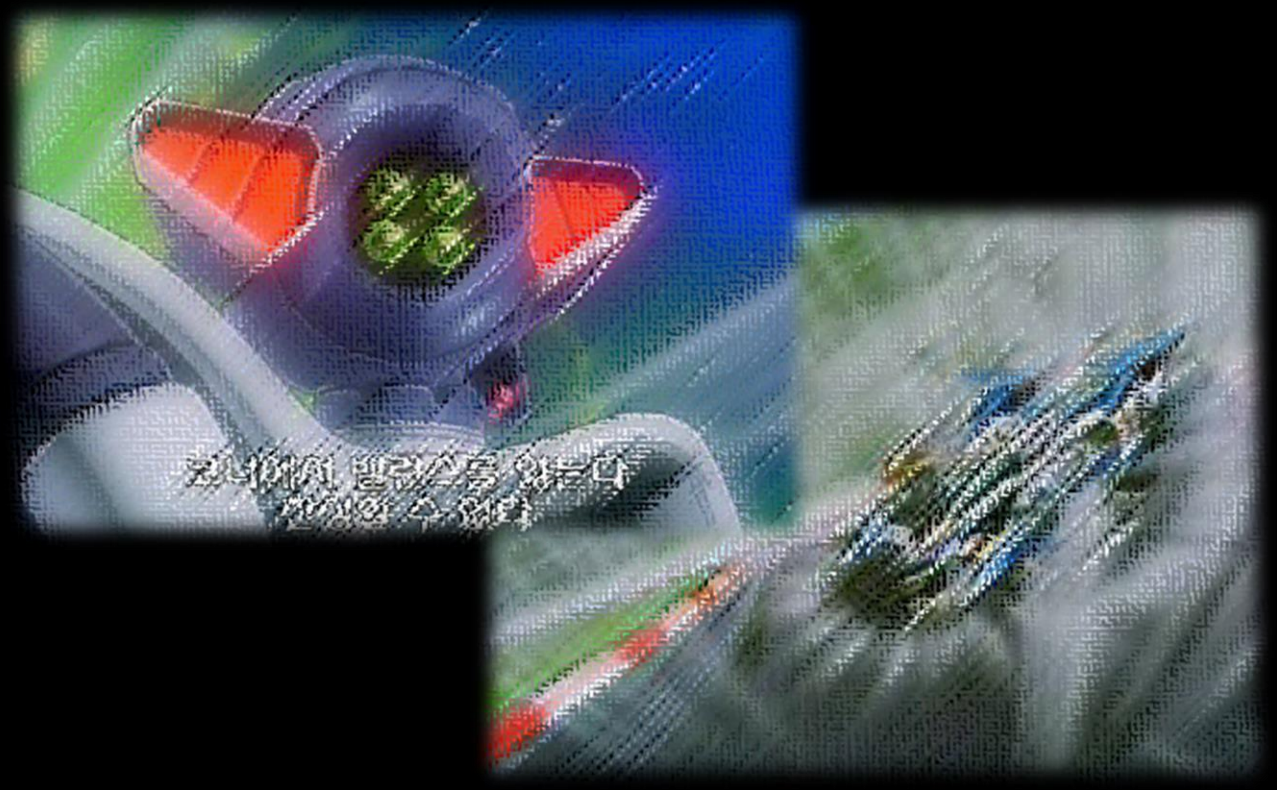

a Priori
권기웅



1. a Priori

1.1 a Priori

1.1.1 학습 데이터

trainData				
A	ㄱ	ㄱ	ㄷ	ㅁ
B	ㄴ	ㄷ	ㅁ	
C	ㄱ	ㄴ	ㄷ	ㅁ
D	ㄴ	ㄹ		

1.1.2 지지도

supportData	
ㄱ	0.5
ㄴ	0.75
ㄷ	0.75
ㄹ	0.25
ㅁ	0.75

- ㄱ의 지지도 = ㄱ이 나온 데이터 수/전체 데이터 4 개 = 2/4 = 0.5

1.1.3 신뢰도

결과	
ㄱ,ㅁ지지도 / ㅁ지지도 = ㅁ -> ㄱ 신뢰도	0.666667
ㄱ,ㅁ지지도 / ㄱ지지도 = ㄱ -> ㅁ 신뢰도	1
...	
ㄱ,ㅁ,ㄷ지지도 / ㄱ지지도 = ㄱ -> ㅁ,ㄷ 신뢰도	1
ㄱ,ㅁ,ㄷ지지도 / ㅁ지지도 = ㅁ -> ㄱ,ㄷ 신뢰도	0.666667
ㄱ,ㅁ,ㄷ지지도 / ㄷ지지도 = ㄷ -> ㄱ,ㅁ 신뢰도	0.666667

만약	
ㄱ,ㄴ,ㄷ,ㄹ,ㅁ/ㄱ,ㄴ = ㄱ,ㄴ->ㄷ,ㄹ,ㅁ 신뢰도	높다면
ㄱ,ㄴ,ㄷ,ㄹ,ㅁ/ㄱ,ㄷ = ㄱ,ㄷ->ㄴ,ㄹ,ㅁ 신뢰도	높다면
ㄱ,ㄴ,ㄷ,ㄹ,ㅁ/ㄴ,ㄷ = ㄴ,ㄷ->ㄱ,ㄹ,ㅁ 신뢰도	
ㄱ,ㄴ,ㄷ,ㄹ,ㅁ/ㄹ,ㅁ = ㄹ,ㅁ->ㄱ,ㄴ,ㄷ 신뢰도	
앞 단계에서 신뢰도 높은것들끼리 두개씩 붙여서 2차전	
ㄱ,ㄴ,ㄷ,ㄹ,ㅁ/ㄴ,ㄷ,ㄹ,ㅁ = ㄴ,ㄷ,ㄹ,ㅁ -> ㄱ	
ㄱ,ㄴ,ㄷ,ㄹ,ㅁ/ㄱ,ㄷ,ㄹ,ㅁ = ㄱ,ㄷ,ㄹ,ㅁ -> ㄴ	
반복	

- ㅁ이 나왔을 때 ㄱ이 나올 신뢰도 = ㅁ,ㄱ이 동시에 나올 지지도 / ㅁ이 나올 지지도 = p(ㄱ| ㅁ) = p(ㅁ && ㄱ) / p(ㅁ)

2. 테스트 데이터로

2.1 APriori.java

2.1.1 데이터 분리하는 메소드

```
// 하나씩 분리
// p : ArrayList<ArrayList<String>> data
//       - [[ㄱ,ㄱ,ㄷ,ㅁ],[ㄴ,ㄷ,ㅁ],[ㄱ,ㄴ,ㄷ,ㅁ],[ㄴ,ㄹ]]
public static ArrayList<HashSet<String>> divideData(ArrayList<ArrayList<String>> 
    ArrayList<HashSet<String>> dividedData = new ArrayList<>();
    HashSet<String> hs = new HashSet<>();
    for (ArrayList<String> al : data) {
        hs.addAll(al);
    }
    String[] ar = hs.toArray(new String[hs.size()]);
    Arrays.sort(ar);
    HashSet<String> hs2 = null;
    for (String i : ar) {
        hs2 = new HashSet<>();
        hs2.add(i);
        dividedData.add(hs2);
    }
    return dividedData; // [{ㄱ}, {ㄴ}, {ㄷ}, {ㄹ}, {ㅁ}]
}
```

2.1.2 중복 아이템 제거하는 메소드

```
// 중복제거
// p : ArrayList<ArrayList<String>> data
//       - [[ㄱ,ㄱ,ㄷ,ㅁ],[ㄴ,ㄷ,ㅁ],[ㄱ,ㄴ,ㄷ,ㅁ],[ㄴ,ㄹ]]
public static ArrayList<HashSet<String>> toSetList(ArrayList<ArrayList<String>> d
    ArrayList<HashSet<String>> setList = new ArrayList<>();
    HashSet<String> hs;
    for (ArrayList<String> al : data) {
        hs = new HashSet<>(al);
        setList.add(hs);
    }
    return setList; // [{ㄱ,ㄷ,ㅁ},{ㄴ,ㄷ,ㅁ},{ㄱ,ㄴ,ㄷ,ㅁ},{ㄴ,ㄹ}]
}
```

2.1.3 최소 지지도 만족하는 데이터만 뽑아내는 메소드

```
// 최소 지지도 만족하는 데이터 구하기(지지도 데이터는 supportData에 저장됨)
public static ArrayList<HashSet<String>> getOKSetList(HashMap<HashSet<String>, Do
        ArrayList<HashSet<String>> setList, // [{ㄱ,ㄷ,ㅁ},{ㄴ,ㄷ,ㅁ},{ㄱ,ㄴ,ㄷ,ㅁ},{ㄴ
        ArrayList<HashSet<String>> dividedData, // [{ㄱ}, {ㄴ}, {ㄷ}, {ㄹ}, {ㅁ}]
        double minSupport) {

    // {ㄱ} : x번 나옴
    // {ㄴ} : x번 나옴
    // ...
    HashMap<HashSet<String>, Integer> setCount = new HashMap<>();
    for (HashSet<String> set : setList) { // {ㄱ,ㄷ,ㅁ}
        for (HashSet<String> d : dividedData) { // {ㄱ}
            if (set.containsAll(d)) { // {ㄱ,ㄷ,ㅁ} 속에 {ㄱ}이 들어있으면
                if (!setCount.containsKey(d)) {
                    setCount.put(d, 1); // {ㄱ} -> 1
                } else {
                    setCount.replace(d, setCount.get(d) + 1);
                }
            }
        }
    }
```

```
    // 최소 지지도 만족하는 데이터만 okSetList에
    // 지지도 데이터는 supportData에
    double numItems = setList.size(); // 4
    ArrayList<HashSet<String>> okSetList = new ArrayList<>();
    Set<HashSet<String>> keySet = setCount.keySet(); // {{ㄱ}, {ㄴ}, {ㄷ}, {ㄹ}, {ㅁ
    double support;
    for (HashSet<String> key : keySet) { // {ㄱ}
        support = setCount.get(key) / numItems; // {ㄱ}이 몇번 나오는지 / 4
        if (support >= minSupport) {
            okSetList.add(key);
        }
        supportData.put(key, support);
    }
    return okSetList; // [{ㄱ},{ㄴ},{ㄷ},{ㅁ}]
}
```

2.1.4 데이터 두 개씩 붙이는 메소드

```java
// 데이터 두개씩 붙이기
public static ArrayList<HashSet<String>> generateAPriori(
        ArrayList<HashSet<String>> okSetList, // [{¬}, {∟}, {⊏}, {⊐}]
        int k) { // 2

    ArrayList<HashSet<String>> newSetList = new ArrayList<>();
    ArrayList<String> al1 = null;
    ArrayList<String> al2 = null;
    Comparator<String> ascending = new Comparator<String>() {
        @Override
        public int compare(String o1, String o2) {
            // TODO Auto-generated method stub
            return o1.compareTo(o2);
        }
    };
```

```java
    int okSetListSize = okSetList.size(); // 4
    HashSet<String> hs = null;
    for (int i = 0; i < okSetListSize; i++) { // 0 ~ 3
        for (int j = i + 1; j < okSetListSize; j++) { // 1 ~ 3
            al1 = new ArrayList<>();
            al2 = new ArrayList<>();
            for (int j2 = 0; j2 < k - 2; j2++) { // x
                al1.add(new ArrayList<String>(okSetList.get(i)).get(j2));
                al2.add(new ArrayList<String>(okSetList.get(j)).get(j2));
            } // al1 : [], al2 : []
            Collections.sort(al1, ascending);
            Collections.sort(al2, ascending);
            if (al1.equals(al2)) {
                hs = new HashSet<>(); // { }
                hs.addAll(okSetList.get(i)); // {¬}
                hs.addAll(okSetList.get(j)); // {∟}
                newSetList.add(hs); // [{¬,∟}]
            }
        }
    }
    return newSetList; // [{¬,∟},{¬,⊏},{¬,⊐},{∟,⊏},{∟,⊐},{⊏,⊐}]
}
```

2.1.5 a Priori 알고리즘 메소드

```
// a Priori 알고리즘
public static ArrayList<ArrayList<HashSet<String>>> apriori(
        HashMap<HashSet<String>, Double> supportData,
        ArrayList<ArrayList<String>> data, // [[ㄱ,ㄱ,ㄷ,ㅁ],[ㄴ,ㄷ,ㅁ],[ㄱ,ㄴ,ㄷ,ㅁ],
        double minSupport) {

    ArrayList<ArrayList<HashSet<String>>> okSetListAll = new ArrayList<>();
    ArrayList<HashSet<String>> dividedData = divideData(data);
    // System.out.println(dividedData); // [{ㄱ}, {ㄴ}, {ㄷ}, {ㄹ}, {ㅁ}]
    ArrayList<HashSet<String>> setList = toSetList(data);
    // System.out.println(setList); // [{ㄱ,ㄷ,ㅁ},{ㄴ,ㄷ,ㅁ},{ㄱ,ㄴ,ㄷ,ㅁ},{ㄴ,ㄹ}]
    ArrayList<HashSet<String>> okSetListFirst = getOKSetList(supportData, setList
    okSetListAll.add(okSetListFirst);

    int k = 2;
    // System.out.println(okSetListAll); //[[{ㄱ},{ㅁ},{ㄴ},{ㄷ}]]
    ArrayList<HashSet<String>> newSetList = null;
    ArrayList<HashSet<String>> newOkSetList = null;
    while (okSetListAll.get(k - 2).size() > 0) {
        newSetList = generateAPriori(okSetListAll.get(k - 2), k);
        newOkSetList = getOKSetList(supportData, setList, newSetList, minSupport)
        okSetListAll.add(newOkSetList);
        k += 1;
    }
    return okSetListAll; // [[{ㄱ}, {ㅁ}, {ㄴ}, {ㄷ}], [{ㄱ, ㅁ},{ㅁ, ㄴ}, {ㄱ, ㄷ}, {
}
```

2.1.6 신뢰도 계산해 결과 만드는 메소드

```java
// 신뢰도 계산하기
public static ArrayList<HashSet<String>> calculateConf(
        HashSet<String> s, // {ㄱ,ㅁ}
        ArrayList<HashSet<String>> setList, // [{ㄱ},{ㅁ}]
        HashMap<HashSet<String>, Double> supportData,
        ArrayList<String> ruleList,
        double minConf) {

    ArrayList<HashSet<String>> prunedSetList = new ArrayList<>();
    double conf;
    String rule;
    HashSet<String> set2;
```

```java
    for (HashSet<String> set : setList) { // {ㄱ}
        set2 = new HashSet<>();
        set2.addAll(s); // {ㄱ,ㅁ}
        set2.removeAll(set); // {ㅁ}

        // {ㄱ,ㅁ}나올 확률 / {ㅁ}나올 확률 = {ㅁ} -> {ㄱ} 신뢰도
        conf = supportData.get(s) / supportData.get(set2);
        if (conf >= minConf) {
            rule = set2 + " --> " + set + " (신뢰도 : " + conf + ")";
            ruleList.add(rule);
            prunedSetList.add(set); // {ㄱ}
        }

        // {ㄱ,ㅁ} 나올 확률 / {ㄱ} 나올 확률 = {ㄱ} -> {ㅁ}
        conf = supportData.get(s) / supportData.get(set);
        if (conf >= minConf) {
            rule = set + " --> " + set2 + " (신뢰도 : " + conf + ")";
            ruleList.add(rule);
            prunedSetList.add(set2); // {ㅁ}
        }
    }
    return prunedSetList; // [{ㄱ},{ㅁ}]
}
```

2.1.7 신뢰도 높은 것 끼리 두 개씩 붙여서 다시 결과 만드는 메소드

```
// 신뢰도 높은것들끼리 두개씩 붙여서 다시 결과 내기
public static void rulesFromResult(
        HashSet<String> s,                  // {¬,▫,⊏}
        ArrayList<HashSet<String>> setList, // [{¬}, {▫}, {⊏}]
        HashMap<HashSet<String>, Double> supportData,
        ArrayList<String> ruleList,
        double minConfidence) {

    int m = setList.get(0).size();  // 1
    ArrayList<HashSet<String>> setList2 = null;
    if (s.size() > (m + 1)) {
        setList2 = generateAPriori(setList, m + 1); // [{¬,▫}, {¬,⊏}, {▫,⊏}]
        setList2
            = calculateConf(s, setList2, supportData, ruleList, minConfidence);
        if (setList2.size() > 1) {
            rulesFromResult(s, setList2, supportData, ruleList, minConfidence);
        }
    }
}
```

2.1.8 최종 결과내는 메소드

```
// 최종 결과 메소드
public static ArrayList<String> generateRules(
        ArrayList<ArrayList<HashSet<String>>> okSetListAll, // [[{¬},{⊔},{⊏},{▫
        HashMap<HashSet<String>, Double> supportData, double minConf) { // 0.5

    ArrayList<String> ruleList = new ArrayList<>();
    ArrayList<HashSet<String>> okSetList = null;
    ArrayList<HashSet<String>> setList = null;
    String[] ar = null;
    HashSet<String> hs = null;
```

```
    for (int i = 1; i < okSetListAll.size(); i++) { // 1
        okSetList = okSetListAll.get(i);            // [{¬,▫},{¬,⊏},{▫,⊔},{▫,

        for (HashSet<String> s : okSetList) {       // {¬,▫}
            setList = new ArrayList<>();            // []
            ar = s.toArray(new String[s.size()]);   // [¬,▫]
            for (String j : ar) {
                hs = new HashSet<>();               // {}
                hs.add(j);                          // {¬}
                setList.add(hs);
            }                                       // [{¬},{▫}]

            if (i > 1) {
                rulesFromResult(s, setList, supportData, ruleList, minConf);
            } else {
                calculateConf(s, setList, supportData, ruleList, minConf);
            }
        }
    }
```

```
    HashSet<String> ruleList2 = new HashSet<>(ruleList);
    Comparator<String> ascending = new Comparator<String>() {
        @Override
        public int compare(String o1, String o2) {
            // TODO Auto-generated method stub
            return o1.compareTo(o2);
        }
    };
    ArrayList<String> ruleList3 = new ArrayList<>(ruleList2);
    Collections.sort(ruleList3, ascending);
    return ruleList3; // [[▫] --> [¬] (신뢰도 : 0.6666666666666666), [¬] --> [▫]
}
```

2.2 APMain.java

2.2.1 테스트 데이터 만드는 메소드

```
// 테스트 데이터 만들기
// r : [[ㄱ,ㄱ,ㄷ,ㅁ], [ㄴ,ㄷ,ㅁ], [ㄱ,ㄴ,ㄷ,ㅁ], [ㄴ,ㄹ]]
private static ArrayList<ArrayList<String>> makeSampleData() {
    ArrayList<ArrayList<String>> trainData = new ArrayList<>();

    ArrayList<String> al = new ArrayList<>();
    al.add("ㄱ");        al.add("ㄱ");        al.add("ㄷ");        al.add("ㅁ");
    trainData.add(al);

    al = new ArrayList<>();
    al.add("ㄴ");        al.add("ㄷ");        al.add("ㅁ");
    trainData.add(al);

    al = new ArrayList<>();
    al.add("ㄱ");        al.add("ㄴ");        al.add("ㄷ");        al.add("ㅁ");
    trainData.add(al);

    al = new ArrayList<>();
    al.add("ㄴ");        al.add("ㄹ");
    trainData.add(al);
    return trainData;
}
```

2.2.2 main()

```
public static void main(String[] args) {
    ArrayList<ArrayList<String>> data = makeSampleData();

    HashMap<HashSet<String>, Double> supportData = new HashMap<>();

    ArrayList<ArrayList<HashSet<String>>> okSetListAll
        = APriori.apriori(supportData, data, 0.5);

    ArrayList<String> ruleList
        = APriori.generateRules(okSetListAll, supportData, 0.5);
    for (String r : ruleList) {
        System.out.println(r);
    }
}
```

2.3 실행

```
[ㄱ, ㄷ] --> [ㅁ] (신뢰도 : 1.0)
[ㄱ, ㅁ] --> [ㄷ] (신뢰도 : 1.0)
[ㄱ] --> [ㄷ] (신뢰도 : 1.0)
[ㄱ] --> [ㅁ, ㄷ] (신뢰도 : 1.0)
[ㄱ] --> [ㅁ] (신뢰도 : 1.0)
[ㄴ, ㄷ] --> [ㅁ] (신뢰도 : 1.0)
[ㄴ] --> [ㄷ] (신뢰도 : 0.6666666666666666)
[ㄴ] --> [ㅁ, ㄷ] (신뢰도 : 0.6666666666666666)
[ㄴ] --> [ㅁ] (신뢰도 : 0.6666666666666666)
[ㄷ] --> [ㄱ, ㅁ] (신뢰도 : 0.6666666666666666)
[ㄷ] --> [ㄱ] (신뢰도 : 0.6666666666666666)
[ㄷ] --> [ㄴ] (신뢰도 : 0.6666666666666666)
[ㄷ] --> [ㅁ, ㄴ] (신뢰도 : 0.6666666666666666)
[ㄷ] --> [ㅁ] (신뢰도 : 1.0)
[ㅁ, ㄴ] --> [ㄷ] (신뢰도 : 1.0)
[ㅁ, ㄷ] --> [ㄱ] (신뢰도 : 0.6666666666666666)
[ㅁ, ㄷ] --> [ㄴ] (신뢰도 : 0.6666666666666666)
[ㅁ] --> [ㄱ, ㄷ] (신뢰도 : 0.6666666666666666)
[ㅁ] --> [ㄱ] (신뢰도 : 0.6666666666666666)
[ㅁ] --> [ㄴ, ㄷ] (신뢰도 : 0.6666666666666666)
[ㅁ] --> [ㄴ] (신뢰도 : 0.6666666666666666)
[ㅁ] --> [ㄷ] (신뢰도 : 1.0)
```

3. 마트 상품 배치하기

3.1 APriori.java

3.1.1 파일 읽어오는 메소드

```
public static ArrayList<ArrayList<String>> trainFromFile(String file) {
    ArrayList<ArrayList<String>> trainData = new ArrayList<>();
    BufferedReader br = null;
    try {
        FileReader fr = new FileReader(file);
        br = new BufferedReader(fr);
        String line = null;
        trainData = new ArrayList<>();
        ArrayList<String> d;
        String[] ar;
        while ((line = br.readLine()) != null) {
            d = new ArrayList<>();
            ar = line.split(" ");
            for (String s : ar) {
                d.add(s);
            }
            trainData.add(d);
        }
        return trainData;
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    } finally {
        try {br.close();} catch (IOException e) {}
    }
}
```

3.2 APMain.java

3.2.1 main()

```
public static void main(String[] args) {
    ArrayList<ArrayList<String>> trainData
        = APriori.trainFromFile("D:\\Kwon\\자원\\for AI/마트.txt");

    HashMap<HashSet<String>, Double> supportData = new HashMap<>();

    ArrayList<ArrayList<HashSet<String>>> okSetListAll
        = APriori.apriori(supportData, trainData, 0.2);

    ArrayList<String> ruleList
        = APriori.generateRules(okSetListAll, supportData, 0.5);
    for (String r : ruleList) {
        System.out.println(r);
    }
}
```

3.3 결과

```
[맥주] --> [치킨] (신뢰도 : 0.605553442373526)
[볼펜] --> [맥주] (신뢰도 : 0.5894788593903638)
[볼펜] --> [치킨] (신뢰도 : 0.6002949852507374)
[사과] --> [맥주] (신뢰도 : 0.5805084745762712)
[사과] --> [치킨] (신뢰도 : 0.6167608286252354)
[사이다] --> [맥주] (신뢰도 : 0.5825559701492538)
[사이다] --> [치킨] (신뢰도 : 0.6161380597014925)
[수박] --> [맥주] (신뢰도 : 0.5798237022526934)
[수박] --> [치킨] (신뢰도 : 0.6082272282076395)
[치킨] --> [맥주] (신뢰도 : 0.5842201834862386)
[컴퓨터] --> [맥주] (신뢰도 : 0.5926456542502389)
[컴퓨터] --> [치킨] (신뢰도 : 0.6069723018147086)
[피자] --> [맥주] (신뢰도 : 0.5680562531391261)
[피자] --> [치킨] (신뢰도 : 0.6047212456052236)
```